Use Cases

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

UC-2 — Log in

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	User goal
Primary actor	Student, Technician or Administrator
Stakeholders and interests	*User*: wants a fast, familiar sign-in without remembering a second credential. *Administration*: wants role-appropriate access enforced from the moment of login.
Preconditions	The user has a registered account.
Success guarantee (postconditions)	The user holds a valid session (JWT) scoped to their role.
Main success scenario	1. User enters their Student/Staff ID and password. 2. System resolves the ID to the account's email (server-side lookup — the client never sees this happen). 3. System authenticates the resolved email + password against Supabase Auth. 4. System returns a session and the user's profile. 5. Client stores the session and routes to the role-appropriate dashboard.
Extensions	3a. Password incorrect, or ID not found: system returns an error; no session issued. 1a. User chooses "Log in with email instead": step 2 is skipped, email is used directly.
Special requirements	The ID→email resolution is a **server-side** step specifically so the client never needs any direct Supabase access (see CLAUDE.md, "Authentication: ID-Based Login").
Technology/data variations	None — one login form serves all three roles; the role returned by the profile determines the UI shown next.
Frequency	Very high — once per session per user.

UC-3 — Submit a maintenance request

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	User goal
Primary actor	Student (or, for UC-38, the Scheduler acting on an administrator's behalf)
Stakeholders and interests	*Student*: wants the fault fixed, with minimal effort to report it. *Administrator*: wants every request classified, located and prioritised well enough to triage without follow-up questions.
Preconditions	Student is logged in.
Success guarantee	A new request exists with status `open`; a `status_history` row records it; the student's hostel is learned for future announcements if not already set.
Main success scenario	1. Student opens "Report New Issue". 2. Student selects a category, hostel, and (optionally) room. 3. Student selects a priority (optionally guided by UC-9's suggestion). 4. Student describes the problem; optionally attaches a photo. 5. Student submits. 6. System validates every field server-side, stores the request as `open`, logs the initial history row, and notifies administrators.

Field	Detail
Extensions	4a. No connectivity: see UC-10 (offline draft) instead of failing. 6a. Validation fails (e.g. missing hostel, description too short): system rejects with a specific 400 error, nothing is stored. 6b. An existing similar report already exists for the same hostel/category: system shows it so the student can choose UC-6 instead.
Special requirements	Photo, if present, is validated (type/size) and uploaded **before** the request row is written, and compressed client-side first so a phone photo (4–10 MB) does not make submission slow on hostel data.
Technology/data variations	Photo may come from the camera (`capture="environment"` / live `getUserMedia` preview) or the file picker — see PhotoInput.jsx.
Frequency	High — the system's core transaction.

UC-9 — Get a triage suggestion while describing a fault

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	Subfunction (of UC-3)
Primary actor	Student
Stakeholders and interests	*Student*: wants help picking the right category/priority without guessing. *Administrator*: wants dangerous faults (fire, flooding) flagged Critical even if the student under-states them.
Preconditions	Student is filling in the request description (≥ 10 characters).
Success guarantee	A suggested category and priority is shown; nothing is saved unless the student explicitly applies it.
Main success scenario	1. Student types a description. 2. System (Claude, if configured) returns a suggested category, priority and a one-line reason. 3. Student taps **Apply** to accept it, or **Dismiss** to ignore it.
Extensions	2a. No `ANTHROPIC_API_KEY`, or the AI call fails: a deterministic keyword-rules engine answers instead, with the identical response shape. 2b. The description contains a hazard keyword (fire, sparking, flooding, gas): priority is forced to `critical` regardless of what either engine would otherwise say — a **safety floor** that cannot be overridden by the model. 1a. More than 10 requests/minute from this user: system returns 429, no suggestion.
Special requirements	Advisory only — the student remains in control (must tap Apply); the description is treated as **untrusted user input** when built into the AI prompt.
Technology/data variations	With vs. without a configured Anthropic API key (two answering engines, one contract).
Frequency	High — offered on every submission.

UC-6 — Back an existing report ("Me too")

Field	Detail
Scope	UNZA Hostel Maintenance System

Field	Detail
Level	User goal
Primary actor	Student
Stakeholders and interests	*Student*: wants to avoid filing a duplicate report. *Administrator*: wants to know how many people a fault affects, to prioritise widely-felt problems.
Preconditions	Student is logged in; the target request is `open` / `assigned` / `in_progress`; the student did not report it themselves.
Success guarantee	An upvote row exists (or is removed, if toggled off); the request's upvote count is accurate; no identity is exposed to other students.
Main success scenario	1. Student sees a matching, already-reported issue in "Already reported in ⟨hostel⟩". 2. Student taps **Me too**. 3. System records the upvote (idempotent — a repeat tap changes nothing extra) and returns the new count.
Extensions	2a. Student already upvoted: tapping again removes their vote instead. 2b. The request is the student's own, or already resolved: the action is refused. 3a. The request reaches 30+ upvotes: it becomes visible to students in **every** hostel, not just its own, flagged "Widespread".
Special requirements	The composite primary key `(request_id, user_id)` on `request_upvotes` makes a duplicate vote impossible even if the service-layer check were bypassed — a database-level guarantee, not just an application one.
Technology/data variations	None.
Frequency	Medium.

UC-16 — Update the status of an assigned request

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	User goal
Primary actor	Technician (also Administrator, who may advance any request)
Stakeholders and interests	*Technician*: wants to record progress accurately. *Student*: wants to know work has started/finished. *Administrator*: wants the lifecycle rule (no skipped or backward steps) enforced without having to police it manually.
Preconditions	The request is assigned to the acting technician (or the actor is an administrator).
Success guarantee	Status advances exactly one step; a `status_history` row is written; the student is notified.
Main success scenario	1. Technician opens an assigned task. 2. Technician presses the one available next-step action (*Start Task* / *Mark as Resolved* / *Close Ticket*). 3. System validates the transition is the single valid next step. 4. System updates status, logs history, notifies the student.
Extensions	3a. A different technician (not assigned) attempts this: refused, 403. 3b. An out-of-order transition is attempted (e.g. skipping straight to Resolved): refused, 400, with a clear message. 2a. Advancing to Resolved: see UC-17 (proof of work) as part of this same action.

Field	Detail
Special requirements	`assigned` is reachable **only** through the assignment action (UC-23), never through this use case directly — status and the assignment record cannot drift apart.
Technology/data variations	None.
Frequency	High.

UC-17 — Attach proof of finished work

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	Subfunction (of UC-16, specifically the → Resolved transition)
Primary actor	Technician
Stakeholders and interests	*Student*: wants evidence the repair actually happened. *Administrator*: wants an optional-by-default, mandatory-if-configured record of completed work.
Preconditions	The request is `in_progress` and assigned to the acting technician.
Success guarantee	If provided, a photo and/or note are stored **before** the status changes to `resolved`; if `REQUIRE_RESOLUTION_PROOF=true`, a photo is mandatory.
Main success scenario	1. Technician marks the task resolved. 2. Technician optionally takes/chooses a photo and writes a note. 3. System validates and stores the photo (if any) **first**. 4. System then advances the status.
Extensions	3a. Photo fails validation (wrong type / too large): rejected before the status changes — a failed upload never silently leaves a request "resolved" with lost proof. 1a. `REQUIRE_RESOLUTION_PROOF=true` and no photo supplied: the resolve action itself is rejected (administrators are exempt from this requirement).
Special requirements	Photo compressed client-side before upload.
Technology/data variations	Camera capture or file picker, same as UC-3.
Frequency	High.

UC-23 — Assign a technician to a request

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	User goal
Primary actor	Administrator
Stakeholders and interests	*Administrator*: wants the best-suited technician chosen quickly. *Technician*: wants a fair, availability-aware workload.

Field	Detail
Preconditions	The request is `open`.
Success guarantee	An assignment record exists; status becomes `assigned`; both student and technician are notified.
Main success scenario	1. Administrator opens Manage Requests. 2. System pre-selects a ★ **recommended** technician (available, then fewest active tasks). 3. Administrator confirms (or picks a different technician) and presses Assign. 4. System creates the assignment and advances status in one operation.
Extensions	3a. The chosen account is not a technician, or the request is not `open`: refused, 400. 2a. No technician is `available`: the ranking still orders by fewest active tasks among `on_job` accounts; an `off_duty` technician is never recommended.
Special requirements	Ranking: `available` < `on_job` < `off_duty`, then fewest active (assigned/in-progress) tasks, then name — a pure, unit-tested function (`users.service.rankTechnicians`).
Technology/data variations	None.
Frequency	High.

UC-10 — Draft a request offline

Field	Detail
Scope	UNZA Hostel Maintenance System (client)
Level	User goal
Primary actor	Student
Stakeholders and interests	*Student*: does not want to lose a report just because hostel Wi-Fi dropped mid-form.
Preconditions	The app shell is already cached (installed or previously visited).
Success guarantee	The draft is never lost; it is submitted automatically, without user action, once connectivity returns.
Main success scenario	1. Student fills the request form while offline. 2. Client detects the network error is a connectivity failure (`isNetworkError`), not a validation error. 3. Client stores the draft in IndexedDB instead of failing the submission outright. 4. Browser fires an `online` event once connectivity returns. 5. Client submits every queued draft through the normal UC-3 flow and removes it from the queue on success.
Extensions	5a. Connectivity drops again mid-sync: the pass stops there rather than burning through the rest of the queue against a dead connection; remaining drafts retry on the next `online` event. 5b. The server genuinely rejects a draft (e.g. a category was deleted): it stays queued and will fail the same way again — a documented, accepted limitation.
Special requirements	Implemented in the page (not the Background Sync API), because Background Sync is unsupported on iOS Safari and Firefox — see ARCHITECTURE.md §8.

Field	Detail
Technology/data variations	IndexedDB (`draftQueue.js`).
Frequency	Low in absolute terms, but exists specifically for the worst-case moment — the whole point of an offline-tolerant NFR.

UC-31 — View recurring-issue detection

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	User goal
Primary actor	Administrator
Stakeholders and interests	*Administrator*: wants to catch a fault that keeps coming back before it becomes a bigger failure, instead of re-fixing the symptom each time.
Preconditions	Administrator is on Reports & Analytics.
Success guarantee	Faults repeating at the same room (3+ in 60 days) or hostel (6+ in 30 days, same category) are surfaced with a suggestion.
Main success scenario	1. System groups recent requests by room+category and hostel+category. 2. System excludes preventive-maintenance-generated requests from the grouping (they are scheduled checks, not organic recurrence). 3. System flags groups past the threshold, `high` severity at double the threshold. 4. Administrator reviews and optionally follows **"Plan a preventive check"** into UC-34, prefilled.
Extensions	1a. Room text varies in casing/spacing ("Block C, Room 24" vs "block c, room 24"): normalised before grouping so they count as the same room.
Special requirements	A pure, unit-tested function (`recurrence.service.detectRecurringIssues`) — same rule, one definition, tested once.
Technology/data variations	None.
Frequency	Low — a periodic review action, not a per-request one.

UC-38 — Generate a request when a preventive-maintenance schedule falls due

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	User goal (system-initiated)
Primary actor	Scheduler (system actor)
Stakeholders and interests	*Administrator*: wants a recurring check (e.g. "inspect the geysers monthly") to become a real, trackable job automatically, without remembering to create it by hand. *Technician*: wants a scheduled check to look and behave like any other task, not a special case.
Preconditions	An active schedule's `next_due_at` has passed.

Field	Detail
Success guarantee	A normal maintenance request exists (created via the **same path** as UC-3), tagged so it can be told apart from a reported fault; the schedule's `next_due_at` has advanced; a new task is never created while the previous one it generated is still open.
Main success scenario	1. Hourly job finds every schedule whose due date has passed. 2. For each: system **advances `next_due_at` first** (so a later failure cannot cause a duplicate fire on the next hourly check). 3. System checks the previous task this schedule generated is finished (not open/assigned/in-progress). 4. System creates the request via `requests.service.createRequest`, prefixing the description `"Preventive maintenance: "`. 5. System records the run and updates `last_generated_at`.
Extensions	3a. The previous task is still open: this schedule is skipped this cycle — no new task is created. 1a. An administrator instead presses **Run now**: steps 2–5 happen immediately, subject to the same "previous task finished" rule. Failures for one schedule are caught so they never block the others in the same batch.
Special requirements	Reuses `requests.service.createRequest` rather than a parallel code path, so the generated request gets a full `status_history`, an SLA, and admin notification exactly like a student's — see LOGICAL_ARCHITECTURE.md §3.
Technology/data variations	None.
Frequency	Low per schedule (weekly to yearly), but many schedules may be checked every hour.

UC-39 — Detect and alert on an overdue request

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	User goal (system-initiated)
Primary actor	Scheduler (system actor); the Administrator and assigned Technician are the notified actors
Stakeholders and interests	*Administrator/Technician*: wants to know a job has blown its SLA without having to check every request manually.
Preconditions	An hourly check runs.
Success guarantee	Every request past its SLA target that has not yet been alerted (or is due a reminder) is alerted exactly once per interval; `overdue_notified_at` reflects the most recent alert.
Main success scenario	1. Every hour, system computes SLA state for every open/assigned/in-progress request. 2. For each newly overdue request: system writes `overdue_notified_at` **before** sending (so a slow/failed send cannot cause a duplicate), then notifies the assigned technician and all administrators (push + live update). 3. For a request still overdue after the reminder interval has elapsed (4 h for critical/high, 24 h for medium/low) since the last alert: repeat step 2, worded "still overdue".
Extensions	2a. More than 3 requests become overdue in the same check: the notifications collapse into one summary rather than flooding the recipient.
Special requirements	SLA state and the reminder rule are pure, unit-tested functions shared with UC-3's SLA badge and UC-30's performance maths — one definition, four uses (`utils/sla.js`).

Field	Detail
Technology/data variations	None.
Frequency	Checked hourly; actually fires only when a request is genuinely overdue.

UC-13 — Submit app feedback

Field	Detail
Scope	UNZA Hostel Maintenance System
Level	User goal
Primary actor	Student or Technician (any authenticated role)
Stakeholders and interests	*Student/Technician*: wants a way to tell the development team something is broken or could be better, without needing the personal contact an administrator already has as a project stakeholder. *Developer*: wants structured, ratable feedback instead of word-of-mouth.
Preconditions	User is logged in.
Success guarantee	A feedback row exists with a rating, category and optional comment, always attributed to the authenticated user; administrators are notified.
Main success scenario	1. User opens Feedback. 2. User picks a 1–5 star rating and a category (bug/suggestion/general). 3. User optionally adds a comment. 4. User submits. 5. System validates, stores it, and notifies administrators (non-blocking).
Extensions	4a. No rating chosen: rejected client-side before submission. 4b. An unrecognised category is sent: system falls back to "general" rather than rejecting the whole submission. 4c. The user has submitted 5+ times in the last hour: rejected, 429.
Special requirements	This is a review of the **app itself**, not of a technician's work — a deliberately separate concept from UC-8 (Rate a finished repair).
Technology/data variations	None.
Frequency	Low.

---

4. Remaining use cases — casual form (≈70%)

Detailed in Elaboration Iteration 2, once the primary use cases above had already proven the architecture. Each is a short, one-scenario description — sufficient because none of these introduced a new architectural risk; they follow patterns already established by §3.

Student

ID	Use case	Summary
UC-1	Register	Student self-registers with name, email, ID number and password; account role defaults to `student`; technician/administrator accounts are provisioned only by an administrator (UC-26).
UC-4	Track my requests	Student views their own requests, filterable by active/finished, searchable by number/description/category.
UC-5	View a request's full detail	Status, priority, assigned technician, description, photo, comments, timeline, and (once resolved) the proof of work and rating prompt.
UC-7	Comment on a request	The reporting student, the assigned technician and administrators — and no one else — can post/read; disabled once `closed`; flood-limited.
UC-8	Rate a finished repair	Once per request, only after resolved/closed, only by the reporting student: 1–5 stars plus an optional comment.
UC-11	Set my hostel	Chosen once in Settings; used to target announcements and to default the "already reported" feed; learned automatically from the student's first request if never set.
UC-12	View announcements	Active notices for the student's hostel, plus every all-hostel notice; dismissible per device.
UC-14	Manage my profile / notifications / theme	Name/photo, push-notification opt-in, light/dark/system theme.

Technician

ID	Use case	Summary
UC-15	View my assigned tasks	Active/completed tabs, search, a deadline badge, an "+N affected" badge, and a "Recurring Maintenance" badge for schedule-generated jobs.
UC-18	Set my availability	Available / On Job / Off Duty — feeds directly into UC-23's ranking.
UC-19	View my performance scorecard	Resolved count, average resolution time, on-time rate, rating, rank among colleagues, over a selectable window.

Administrator

ID	Use case	Summary
UC-22	Manage / triage all requests	Every request, filterable by status (including an Overdue tab), searchable; deep-links from the dashboard's Overdue tile.
UC-24	Change request priority	Any request, any time, independent of status transitions.
UC-25	Advance or close a request directly	An administrator may advance any request's status, bypassing the "must be the assigned technician" rule UC-16 applies to technicians.
UC-26	Provision a staff account	Name, staff ID, email, temporary password, role (technician/administrator); students cannot be created here.